

SIR YASIR ALI LASHARI

TITANS-SUKKUR

Node.js Backend Notes — Login System, Password Safety, JWT & File Upload (Multer)

List of Topics Covered

01	Express.js Server Setup
02	Connecting to MongoDB (Mongoose)
03	CORS (Letting Frontend Talk to Backend)
04	Environment Variables (dotenv)
05	Mongoose Schema / Model (User Data Shape)
06	Password Hashing (bcryptjs)
07	User Registration (Sign Up)
08	User Login (Sign In)
09	JWT (JSON Web Token) — Making & Checking
10	Uploading Images (Multer)
11	API Routes
12	Error Handling (try/catch)

1 Password Hashing (bcryptjs)

What It Means

Password hashing means turning a normal password into a secret code before saving it in the database. A hash cannot be turned back into the real password — it only goes one way. In this code, `bcrypt.hash(password, 10)` does this job. The number **10** is called **salt rounds**. A bigger number makes the hash safer, but a little slower to make.



Fig 1.1 — How a normal password turns into a safe hash

Real-Life Example

Think about a **paper shredder**. Once you shred a paper, it turns into small pieces. You cannot put those pieces back together to read the paper again. But if you shred the same paper again, it will look the same way. This is how bcrypt checks your password at login — it shreds the typed password again and checks if it matches the saved shredded version, using `bcrypt.compare(password, user.password)`.

Benefits

- Even if someone steals the database, they cannot see the real passwords.
- The **salt** makes sure two same passwords get two different hashes.
- It stops easy hacking tricks like rainbow tables.
- Almost every real app in the world uses this method.

2 JWT – JSON Web Token

What It Means

A JWT is a small piece of text that proves who you are. The server gives it to you after login, so you don't need to type your password again and again. It is made using `jwt.sign(payload, process.env.JWT_SECRET, { expiresIn: "1h" })`. It has three parts joined by dots: **Header**, **Payload**, and **Signature**.

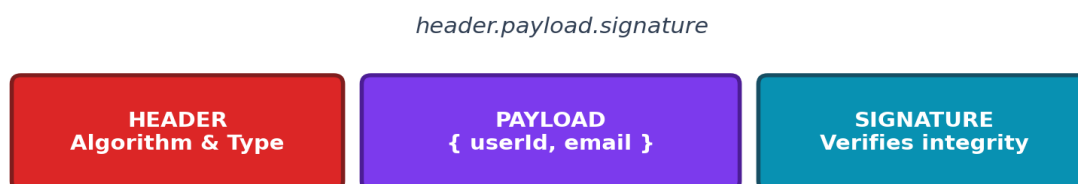


Fig 2.1 — The three parts of a JWT token

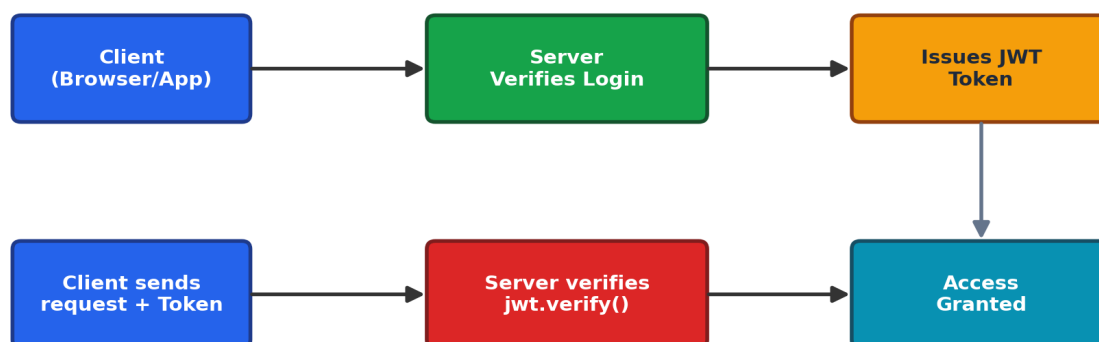


Fig 2.2 — How JWT works between the app and the server

Real-Life Example

A JWT is like a **wristband at a concert**. The guard checks your ticket only one time at the gate. After that, you just show your wristband to enter any other gate — no need to show your ticket again. The wristband also stops working after the event ends, just like this token stops working after **1 hour**.

Benefits

- The server does not need to remember every user — it just checks the token.
- The token itself holds the user's info (userId, email).
- No one can change the token without breaking it.
- A short time limit (**1h**) keeps things safer if it gets stolen.
- Works great for websites, mobile apps, and small services.

3 Authentication (Register & Login)

What It Means

Authentication means checking **who the user is**. This code has two routes for this: `/register` (makes a new user and saves the hashed password) and `/login` (checks the password and gives a JWT token if it is correct).



Fig 3.1 — Register → Hash → Save → Login steps

Real-Life Example

This is like **opening a bank account**. When you open an account (register), the bank keeps your details safe and never writes your PIN in plain words anywhere. Every time you visit the bank again (login), the staff checks your details before letting you use your account.

Benefits

- Stops other people from using someone else's account.
- Checking for the same email stops two users from signing up twice.
- Sign up and login code is kept separate and easy to understand.
- Makes it easy to add more security later, like protected pages.

4 Multer – File / Image Upload

What It Means

Multer is a helper tool that lets your server accept file uploads, like images. Normal Express code cannot read files by itself, so we use Multer. In this code, `multer.diskStorage()` saves images into a folder called `uploads/`. Each file name starts with `Date.now()` so two files never get the same name. Then `upload.single("image")` handles one image on the `/api/upload/single` route.



Fig 4.1 — How Multer saves an uploaded image

Real-Life Example

Think of a **hotel reception desk with a luggage room**. When a guest brings a bag (a file), the reception (Multer) puts a ticket number and time on it, so it never gets mixed up with another bag. Then it keeps the bag safe in the luggage room (`uploads/` folder). The guest gets a small ticket (`req.file`) to find the bag again later.

Benefits

- Lets the server accept files, which normal Express code cannot do alone.
- Gives every file a clean, unique name automatically.
- Can handle one file or many files at once.
- Can be set up to block very big files or wrong file types (using `fileFilter` and `limits`).
- Can also be connected to cloud storage like S3 or Cloudinary later.

5 Complete Backend Code

Below is the full code, file by file, exactly as it was given.

Packages to Install First

Before running this code, open your project folder in the terminal and install these packages:

```
npm init -y
```

```
npm install express mongoose cors dotenv bcryptjs jsonwebtoken multer
```

- **express** — runs the web server
- **mongoose** — connects to MongoDB
- **cors** — lets your frontend talk to this backend
- **dotenv** — reads secret values from the .env file
- **bcryptjs** — makes password hashes
- **jsonwebtoken** — makes and checks JWT tokens
- **multer** — handles image/file uploads

server.js

```
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");
const authRoutes = require("./routes/auth");
const uploadRoutes = require("./routes/upload");
require("dotenv").config();
const app = express();

// ===== MIDDLEWARE =====
app.use(cors());
app.use(express.json());

// ===== MONGODB =====
mongoose
  .connect(process.env.MONGO_URI)
  .then(() => {
    console.log("MongoDB Connected");
  })
  .catch((err) => {
    console.error("MongoDB Error:");
    console.error(err.message);
  });

// ===== ROUTES =====
app.use("/api/auth", authRoutes);
app.use("/api/upload", uploadRoutes);

// ===== SERVER =====
app.listen(5000, () => {
  console.log("Server running on port 5000");
});
```

models/User.js

```
const mongoose = require("mongoose");

const userSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: true,
    },
    email: {
      type: String,
      required: true,
      unique: true,
    },
    password: {
      type: String,
      required: true,
    },
    profileImage: {
      type: String,
      default: "",
    },
  },
  {
    timestamps: true,
  }
);

module.exports = mongoose.model("User", userSchema);
```

routes/auth.js

```
// Install JWT first:
// npm install jsonwebtoken
const router = require("express").Router();
const User = require("../models/User");
const bcrypt = require("bcryptjs");
const jwt = require("jsonwebtoken");

// ===== REGISTER =====
router.post("/register", async (req, res) => {
  try {
    const { name, email, password } = req.body;

    const exist = await User.findOne({ email });
    if (exist) {
      return res.status(400).json({ message: "User already exists" });
    }

    const hash = await bcrypt.hash(password, 10);

    const user = new User({
      name,
      email,
      password: hash,
    });

    await user.save();
    res.status(201).json({ message: "Registration Successful" });
  } catch (err) {
    res.status(500).json({ message: "Server Error" });
  }
});
```

```

    }
  });

  // ===== LOGIN =====
  router.post("/login", async (req, res) => {
    try {
      const { email, password } = req.body;

      const user = await User.findOne({ email });
      if (!user) {
        return res.status(404).json({ message: "User not found" });
      }

      const isMatch = await bcrypt.compare(password, user.password);
      if (!isMatch) {
        return res.status(401).json({ message: "Invalid email or password" });
      }

      const payload = {
        userId: user._id,
        email: user.email,
      };

      const token = jwt.sign(payload, process.env.JWT_SECRET, {
        expiresIn: "1h",
      });

      res.status(200).json({
        message: "Login Successful",
        token: token,
        user: {
          id: user._id,
          name: user.name,
          email: user.email,
        },
      });
    } catch (err) {
      res.status(500).json({ message: "Server Error" });
    }
  });

  module.exports = router;

```

routes/upload.js

```

const router = require("express").Router();
const multer = require("multer");

// ===== STORAGE =====
const storage = multer.diskStorage({
  destination: function (req, file, cb) {
    cb(null, "uploads/");
  },
  filename: function (req, file, cb) {
    cb(null, Date.now() + "-" + file.originalname);
  },
});

// ===== MULTER =====
const upload = multer({
  storage: storage,
});

```



```
// ===== SINGLE FILE UPLOAD =====
router.post("/single", upload.single("image"), (req, res) => {
  try {
    res.status(200).json({
      message: "File uploaded successfully",
      file: req.file,
    });
  } catch (error) {
    res.status(500).json({
      message: "File upload failed",
      error: error.message,
    });
  }
});

module.exports = router;
```

package.json

```
{
  "name": "backend",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs",
  "dependencies": {
    "bcryptjs": "^3.0.3",
    "cors": "^2.8.6",
    "dotenv": "^17.4.2",
    "express": "^5.2.1",
    "jsonwebtoken": "^9.0.3",
    "mongoose": "^9.9.1",
    "multer": "^2.2.0"
  }
}
```

Note: The **.env** file (MongoDB link and JWT secret) is not shown here to keep it safe.

6 Testing with Postman

Postman is a free app used to test API routes without building a frontend. Here is how to test all three routes of this backend, step by step.

Step 1 — Register a New User

POST `http://localhost:5000/api/auth/register`

- Open Postman and create a new request.
- Set the method to **POST** and paste the URL above.
- Click the **Body** tab, then choose **raw**, then choose **JSON** from the dropdown.
- Type the JSON shown below, then click **Send**.

```
{
  "name": "Muhammad",
  "email": "Muhammad@gmail.com",
  "password": "Muhammad123"
}
```

If it works, you will get a message that says **Registration Successful**.

Step 2 — Login with the Same User

POST `http://localhost:5000/api/auth/login`

- Create another new request, set the method to **POST**, and paste the URL above.
- Click **Body** tab → choose **raw** → choose **JSON**.
- Type the JSON shown below, then click **Send**.

```
{
  "email": "Muhammad@gmail.com",
  "password": "Muhammad123"
}
```

If it works, the response will include a **token**. Copy this token — you will need it later if you build protected routes.

Step 3 — Upload an Image

POST `http://localhost:5000/api/upload/single`

- Create another new request, set the method to **POST**, and paste the URL above.
- Click the **Body** tab, then choose **form-data** (not raw this time).
- In the **Key** column, type **image**.
- On the right side of the key, change the type from **Text** to **File**.
- Click **Select Files** and choose an image from your computer.
- Click **Send**.

If it works, you will get a message that says **File uploaded successfully**, along with the file details. The image will also be saved inside the **uploads/** folder in your project.